Intro

Let the interpolation nodes be

$$a = t_0 < t_1 < \cdots < t_n = b$$

Instead of one polynomial of degree n, we construct a different polynomial p_j on each interval:

$$s(x) = p_j(x) \qquad \text{with} \quad x \in [t_j, t_{j+1}]$$

And interval lengths are $h_j = t_{j+1} - t_j$

Smoothness C^0, C^1 and C^2

Two polynomials meet at a node t_j:

- C_0: The pieces meet without a jump, so the function is continuous
- C_1: The pieces meet and their first derivatives agree, so there is no abrupt change in tangent direction or derivative.
- C_2: The function values, first and second derivatives agree at each interior knot

The normalized local coordinate

Throughout the session we will frequently use

$$\tau = \frac{x - t_j}{h_j}, \qquad x \in [t_{j+1}, t_j]$$

Observe that this transfers every interval, regardless of its actual length, into the standard interval $[0, 1]$.

→ We do this because we can derive a formula on $[0, 1]$ and then use it on every interval!

Piecewise Linear Interpolation

Given two points (t_j, y_j) and (t_{j+1}, y_{j+1}) the linear interpolant is:

$$s_j(x) = (1-\tau)\, y_j + \tau\, y_{j+1} \qquad x \in [t_j, t_{j+1}]$$

This is a weighted avg. of the endpoint values.

The derivative:

$$\Rightarrow s_j(x) = y_j - \tau\, y_j + \tau\, y_{j+1}$$

$$\Leftrightarrow s_j(x) = y_j - \frac{x-t_j}{h_j}\, y_j + \frac{x-t_j}{h_j}\, y_{j+1}$$

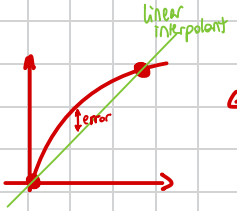
$$\Rightarrow s_j'(x) = 0 - (1/h_j)\, y_j + (1/h_j)\, y_{j+1}$$

$$\Leftrightarrow s_j'(x) = \frac{y_{j+1} - y_{j+1}}{h_j}$$

This is constant, so generally piecewise linear interpolant is C^0 not C^1

Error and Convergence

For a polynomial of degree **at most n** interpolating f at $n+1$ nodes;



$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^{n} (x - x_i)$$

→ errors between interp. nodes

↳ captures how much the function deviates from the behaviour of a degree-n poly.

This holds if f is sufficiently differentiable.

For linear interpolation we have a poly of degree 1, using two nodes t_j and t_{j+1}:

$$f(x) - s_j(x) = \frac{f''(\xi_x)}{2} (x - t_j)(x - t_{j+1})$$

$$\Rightarrow \|f - s\|_\infty \leq \frac{h^2}{8} \|f''\|_\infty \quad \dots \text{ error is } O(h^2)$$

→ $h = \max_j h_j$

Intuition: See that the error depends on the curvature of f. If it is straight, linear interpolant is accurate. But if it bends rapidly we have a problem.

What is a spline?

A spline of degree m is a function $s: [a,b] \to \mathbb{R}$ s.t:

1. On every $[t_j, t_{j+1}]$, s is a poly of degree at most m.
2. The poly. pieces satisfy continuity conditions at the nodes.

For the standard spline space in the lecture we additionally require:

$$s \in C^{m-1}([a,b])$$

We can formally write

$$S_{m,G} = \{ s \in C^{m-1}([a,b]) : s|_{[t_j, t_{j+1}]} \in P_m \}$$

Here $G = \{t_0, \dots, t_n\}$ denotes the grid, P_m is the space of poly. of degree at most m and $s|_{[t_j, t_{j+1}]}$ means the restriction of s to that interval.

Cubic Hermite interpolation versus cubic splines

Cubic Hermite interpolation

Suppose we know not only $y_j = f(t_j)$, but also the derivatives $c_j = f'(t_j)$.

On an interval $[t_j, t_{j+1}]$ we want a cubic poly to satisfy four conditions:

$$s_j(t_j) = y_j \; ; \; s_j(t_{j+1}) = y_{j+1} \; ; \; s_j'(t_j) = c_j \; ; \; s_j'(t_{j+1}) = c_{j+1}$$

We will use τ now and write our cubic poly. as

$$s_j(x) = P_j(\tau) = a_j + b_j \tau + d_j \tau^2 + e_j \tau^3 \quad (*)$$

$$\Rightarrow s_j'(x) = \frac{1}{h_j} (b_j + 2 d_j \tau + 3 e_j \tau^2)$$

- Apply $x = t_j$ we have $\tau = 0$:

$$s_j(t_j) = a_j = y_j$$

and

$$s_j'(t_j) = \frac{b_j}{h_j} = c_j \quad \Rightarrow \quad b_j = h_j c_j$$

- Apply $x = t_{j+1}$ we have $\tau = 1$:

$$s_j(t_{j+1}) = a_j + b_j + d_j + e_j = y_{j+1} \qquad (A)$$

and

$$s_j'(t_{j+1}) = \frac{1}{h_j}(b_j + 2d_j + 3e_j) = c_{j+1} \qquad (B)$$

Substitute $a_j = y_j$ and $b_j = h_j c_j$ into (A)

$$y_j + h_j c_j + d_j + e_j = y_{j+1}$$
$$\Rightarrow \quad d_j + e_j = y_{j+1} - y_j - h_j c_j$$

Similarly do (B)

$$\frac{1}{h_j}(h_j c_j + 2d_j + 3e_j) = c_{j+1}$$

$$\Rightarrow \quad 2d_j + 3e_j = h_j c_{j+1} - h_j c_j$$

Now we have d_j, e_j as unknown and we have 2 conditions hence we solve the system:

$$\begin{pmatrix} 1 & 1 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} d_j \\ e_j \end{pmatrix} = \begin{pmatrix} y_{j+1} - y_j - h_j c_j \\ h_j (c_{j+1} - c_j) \end{pmatrix}$$

$$\Rightarrow \quad d_j = 3(y_{j+1} - y_j) - h_j(2c_j + c_{j+1})$$
$$e_j = 2(y_j - y_{j+1}) + h_j(c_j + c_{j+1})$$

We substitute into $(\star)$ and group the terms acc. to $y_j, y_{j+1}, c_j, c_{j+1}$:

$$P_j(\tau) = \underbrace{(1 - 3\tau^2 + 2\tau^3)}_{H_{00}} y_j + \underbrace{(3\tau^2 - 2\tau^3)}_{H_{01}} y_{j+1} + h_j \underbrace{(\tau - 2\tau^2 + \tau^3)}_{H_{10}} c_j + h_j \underbrace{(\tau^3 - \tau^2)}_{H_{11}} c_{j+1}$$

- C^1 with $s_{j-1}(t_j) = s_j(t_j)$, $s_{j-1}'(t_j) = s_j'(t_j)$
- In practice we don't know derivatives $f'(t_j) = c_j$, so must choose them ourselves given the data.

PCHIP is useful for this, It prioritizes respecting the data rather than finding most accurate approximation the true derivatives

is a method (Piecewise Cubic Hermite Interpolating Polynomial)

Also know its error behaves as $O(h^4)$

Return to Splines

A small example:

$(x_0, y_0) = (0,0)$ $(x_1, y_1) = (1,1)$ $(x_2, y_2) = (2,0)$

We have two intervals $[0,1]$ and $[1,2]$. We want to construct a cubic spline.

$$p_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3$$

$$p_2(x) = a_2 + b_2 x + c_2 x^2 + d_2 x^3$$

$\Rightarrow$ There are 8 unknowns.

$\Rightarrow$ With $p_1(0) = 0$, $p_1(1) = 1$, $p_2(1) = 1$, $p_2(2) = 0$ we can write 4 conditions

Also now we want C^2 (by def of splines above)

$\Rightarrow$ $p_1'(1) = p_2'(1)$ and $p_1''(1) = p_2''(1)$ ... 2 more conditions

- $8 - 6 = 2$ so we need two additional conditions

In general with N intervals and poly of degree at most d we have $N(d+1)$ unknowns.

Remark 4.1.3 $\dim S_{d,G} = N(d+1) - (N-1)d = N + d$

$\Rightarrow$ So we need $N+d$ indep parameters to describe a spline in $S_{d,G}$.

For cubic spline we have $N+3$, since we have $N+1$ interpolation nodes we impose $N+1$ interpolation conditions.

- $(N+3) - (N+1) = \underline{2}$

Thus every standard cubic spline interp. problem requires two additional indep. conditions to obtain an unique solution.

Boundary conditions

These tell us how the curve should behave at its endpoints. ([a, b])

Different choice of boundary cond. generally produce different interp. curves.

• Complete (clamped) cubic spline

$$s'(a) = \alpha \quad \text{and} \quad s'(b) = \beta$$

Intuition: We know the direction (slope) in which the curve starts and finishes

This is useful when reliable derivative information is available.

• Natural Cubic Spline

$$s''(a) = 0 \quad \text{and} \quad s''(b) = 0$$

Intuition: At the endpoints, the graph has zero curvature

This is useful when endpoint derivatives are unavailable and we want a principled choice that avoids unnecessary bendings.

• Periodic Cubic Spline

$$s(a) = s(b) \quad \text{and} \quad s'(a) = s'(b) \quad \text{and} \quad s''(a) = s''(b)$$

(→ conditions)

↳ so $y_0 = y_N$ already in data (is an assumption)

• Not-a-knot condition

Normally a cubic spline is only required to be C^2. So its 3rd derivativ may therefore jump at interior knots.

$$s_0'''(t_1) = s_1'''(t_1) \quad \text{and} \quad s_{N-2}'''(t_{N-1}) = s_{N-1}'''(t_{N-1})$$

Since third derivative of a cubic poly. is constant we get $s_0 = s_1$, and $s_{N-2} = s_{N-1}$ (and C^2)

→ This is default b.c. in SciPy's CubicSpline

Splines lead to linear systems. We know look at how to compute the spline in practice.

→ Naive approach is to solve for all $4N$ poly coeffs, not efficient.

Instead of solving for four coeffs per interval, we can solve directly for the unknown slopes

$$c_j = s'(t_j) \qquad j = 0, \dots, N$$

There are only $N+1$ unknowns.

The Cubic Hermite formula uses these slopes to construct each polynomial segment. Adjacent segments share these values ⇒ piecewise poly is C^1

→ We still need to impose C^2:

Take $[t_{j-1}, t_j]$ and $[t_j, t_{j+1}]$ ⇒ $h_{j-1} = t_j - t_{j-1}$, $h_j = t_{j+1} - t_j$

Define slopes:

$$\delta_{j-1} = \frac{y_j - y_{j-1}}{h_{j-1}} \qquad \delta_j = \frac{y_{j+1} - y_j}{h_j}$$

general

Diff $P_j(\tau)$ twice:

$$P_j(\tau)'' = \frac{12\tau - 6}{h_j^2} y_j + \frac{-12\tau + 6}{h_j^2} y_{j+1} + \frac{6\tau - 4}{h_j} c_j + \frac{6\tau - 2}{h_j} c_{j+1}$$

↓ $\frac{ds_j}{dx} = \frac{dp_j}{d\tau} \cdot \frac{d\tau}{dx}$ and $\frac{d^2 s_j}{d^2 x} = \frac{d}{dx}\left(\frac{dp_j}{d\tau} \cdot \frac{d\tau}{dx}\right) = \frac{1}{h_j^2} p_j''(\tau)$

$$s_j''(t_j) = \frac{6\delta_j - 4c_j - 2c_{j+1}}{h_j} \quad ; \quad s_j''(t_{j+1}) = \frac{-6\delta_j + 2c_j + 4c_{j+1}}{h_j} \qquad (2)$$

Apply this to our intervals and as we require C^2 we require $s_{j-1}''(t_j) = s_j''(t_j)$; rearranging gives:

$$\frac{c_{j-1}}{h_{j-1}} + 2\left(\frac{1}{h_{j-1}} + \frac{1}{h_j}\right) c_j + \frac{c_{j+1}}{h_j} = 3\left(\frac{\delta_{j-1}}{h_{j-1}} + \frac{\delta_j}{h_j}\right) \qquad (1)$$

only involves c_{j-1}, c_j and c_{j+1} as unknown slopes

small example continued:

Recall $(0,0)$, $(1,1)$, $(2,0)$. Grid is equally spaced so $h_0 = h_1 = 1$. and $\delta_0 = 1$, $\delta_1 = -1$. We dont know c_0, c_1, c_2.

At the single interior knot $(1,1)$ the C^2 equation (1) becomes:

$$c_0 + 4c_1 + c_2 = 3(1-1) = 0 \rightarrow$$

3 unknowns
1 equation

you can pick diff. cond and see we reach a tridiagonal matrix regardless!

Choose natural boundary conditions: $S''(0) = S''(2) = 0$

Using (2) we get 2 more equations/conditions:

$$2c_0 + c_1 = 3 \quad \text{and} \quad c_1 + 2c_2 = -3$$

So we have a system:

$$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ -3 \end{bmatrix} \Rightarrow \begin{array}{l} c_0 = 3/2 \\ c_1 = 0 \\ c_2 = -3/2 \end{array}$$

We now know the function values and der. at all three nodes. That is enough to construct the two cubic poly pieces using the Hermite formula.

Extra:

$$\begin{bmatrix} * & * & 0 & \cdots & 0 \\ 0 & * & * & * & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & \cdots & 0 & * & * \\ 0 & \cdots & 0 & * & * \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_N \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ \vdots \\ r_N \end{bmatrix}$$

This is a tridiagonal matrix. This system can be solved in $O(N)$ operations using Thomas algorithm.

Code expert: Lineare Interpolation

We are given (t_i, y_i) $i = 1, \dots, n$ and must implement a Python class representing their piecewise linear interpolant. The class must evaluate the interpolant on a grid of arbitrary query points.

Math. intuition:

On each $[t_i, t_{i+1}]$ the interpolant is:

$$s(x) = y_i + \underbrace{\frac{y_{i+1} - y_i}{t_{i+1} - t_i}}_{\text{= slope}} (x - t_i)$$

Slope does not depend on query points!

Coding Tips

- Precompute slopes in `__init__`, `np.diff()` might be useful!
- For a query point x we need to determine, which interval contains it. → Do this efficiently
- Avoid invalid indexing at the boundaries: Nodes $[0, 1, 2, 3]$ and query point $x = 4$ → disallow it
- Error $(h) \approx Ch^2$ so $\log E(h) = \log C + 2 \log h$ → slope 2 in log-log-plot!

Code Expert: Konvergenzstudie: Kubische Hermite und Splines

Use SciPy constructors:

PchipInterpolator, CubicHermiteSpline and CubicSpline

Try natural, clamped, not-a-knot and periodic for CubicSpline

```
# Define the interpolators to test
# a dictionary "Name_of_Interpolator": constructor
# e.g.
#  "CubicSpline (natural)": lambda x, y, **kwargs: CubicSpline(x, y, bc_type='natural'),
# watch the documentation and below how they are  used
interpolators: Dict[str, Any] = {
  # TODO: ergänzen Sie mit dem richtigen Code

}
```

Code expert: Geschlossene Kurveninterpolation mit quadratischen Splines

Previously we interpolated a scalar function $f: [a,b] \to \mathbb{R}$

Now we want to reconstruct a curve in the plane:

$$s: [0,1] \to \mathbb{R}^2, \quad s(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$$

Parameter t describes our position along the curve.

Chord-length parametrization

distinct vertices = n

We are given a sequence of points $p^0, p^1, \dots, p^{n-1}$ $p^n = p^0$

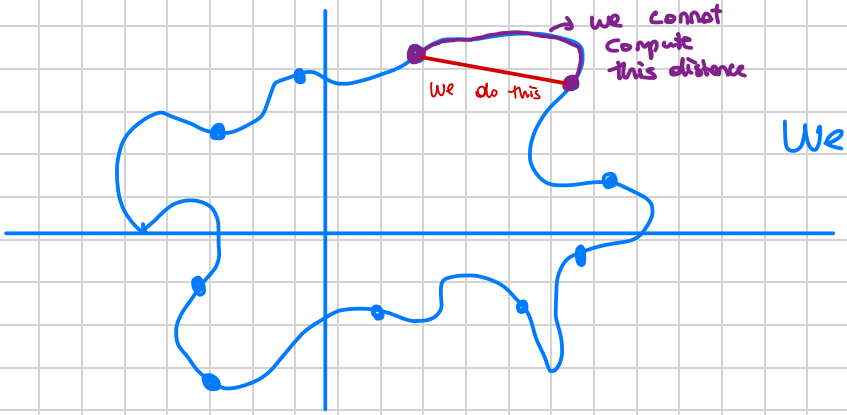
The curve must pass through these points in the specified order

$\Rightarrow$ So $s(t)$ must fulfil $s(t_0) = p^0$, $s(t_1) = p^1$ ....

What should the parameter values $t_0, t_1, \dots, t_n$ be?

- $t_j = j/n$ -- equally spaced, but points could be not equally spaced

A chord is the straight line segment connecting two points on a curve.



We only know •

For two consecutive points we have $l_j = \| p^j - p^{j-1} \|_2$ and we define $L_{poly} = \sum_{k=1}^{n} l_k$ -- total length of polygon defined by line segments

$$\Rightarrow t_j = \frac{\sum_{k=1}^{j} l_k}{\sum_{k=1}^{n} l_k}$$ ... we normalize it to describe our curve on the interval $[0,1]$.

Coding Tips

- np.diff() and np.cumsum()

Constructing a quadratic spline

- We want piecewise quadratic curve $s: [0,1] \to \mathbb{R}^2$ satisfying $s(t_j) = p^j$.
- Additionally, we want the curve to be C^1

Step 1:

On the interval $[t_{j-1}, t_j]$ we have $\tau = \frac{t - t_{j-1}}{h_{j-1}}$

Linear interpolant is $L_j(t) = (1-\tau)p^{j-1} + \tau p^j$, this satisfies interpolation condition trivially.

↳ We still need C^1 but line segments have different slopes and when we connect them we have corners.

Step 2:

Allow the line segment to bend!

We add a quadratic correction $q_j(\tau)$ w/o changing endpoints.

⇒ we require $q_j(0) = q_j(1) = 0$.

⇒ General quadratic poly is: $q_j(\tau) = A_j + B_j \tau + C_j \tau^2$ (A_j, B_j, C_j in $\mathbb{R}^2$)

⇒ From the conditions we get: $A_j = 0$ and $B_j + C_j = 0$

⇒ Therefore $q_j(\tau) = B_j \tau(1-\tau)$

any quadratic correction that we will use must be multiple of $\tau(1-\tau)$

$$\Rightarrow s_j(t) = (1-\tau)p^{j-1} + \tau p^j + B_j \tau(1-\tau)$$

$$\Rightarrow s_j'(t) = \frac{p^j - p^{j-1}}{h_{j-1}} + \frac{B_j}{h_{j-1}}(1-2\tau)$$

$\frac{B_j}{h_{j-1}}$ → let this be χ^j

So χ^j determines how the derivatives at the endpoints deviate from straight-chord slope!

⇒ To obtain C^1 we impose $s_j'(t_j) = s_{j+1}'(t_j)$, instead of working seperately with left and right derivatives we define one shared vector at each knot:

$$d_j = s'(t_j)$$

On interval $[t_{j-1}, t_j]$ we have d_{j-1} and d_j and they are equal to $\frac{p^j - p^{j-1}}{h_j} + x^j$, $\frac{p^j - p^{j-1}}{h_j} - x^j$ resp.

→ Here both d's and x^j is unknown, we add d_{j-1} and d_j to get:

$$d_{j-1} + d_j = \frac{2}{h_j}(p^j - p^{j-1}) \qquad j = 1, \dots, n$$

main equation in the exercise

→ And for x^j's do $d_{j-1} - d_j$ to get $x^j = \frac{1}{2}(d_{j-1} - d_j)$

Strategy: First find d's and then compute x^j's

From our derivative equations to a Linear System

Recall what we have already derived:

$$d_j + d_{j+1} = \frac{2}{h_j}(p^{j+1} - p^j)$$

Let $r_j = \frac{2}{h_j}(p^{j+1} - p^j)$ and as the curve is closed $p^n = p^0$; $d_n = d_0$.

$$\Rightarrow \begin{aligned} d_0 + d_1 &= r_0 \\ d_1 + d_2 &= r_1 \\ &\vdots \\ d_{n-1} + d_0 &= r_{n-1} \end{aligned}$$

We write this as a cyclic linear system:

$$\underbrace{\begin{bmatrix} 1 & 1 & 0 & \cdots & 0 \\ 0 & 1 & 1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 1 \\ 1 & 0 & \cdots & 0 & 1 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} d_0 \\ d_1 \\ \vdots \\ d_{n-1} \end{bmatrix}}_{D} = \underbrace{\begin{bmatrix} r_0 \\ r_1 \\ \vdots \\ r_{n-1} \end{bmatrix}}_{R}$$

- build_A

As $d_j, r_j \in \mathbb{R}^2$
$\Rightarrow D, R \in \mathbb{R}^{n \times 2}$

⇒ This can give you d's with a method we used earlier

Curvature

$$\kappa(t) = \frac{x'(t)\,y''(t) - y'(t)\,x''(t)}{\left(x'(t)^2 + y'(t)^2\right)}$$

already given in the task

⇒ For our quadratic spline look at $s_j'(x)$ and $s_j''(x)$ and remember that $s_j(x) = \begin{pmatrix} x_j(x) \\ y_j(x) \end{pmatrix}$

Coding Tips:

- use self.derivative() for derivation
- for length() look at np.trapz

Code expert: Adaptive Interpolation geschlossener Kurven

Notation of the exercise

- $c(t)$: the original closed curve
- $\Sigma = (p_1, \dots, p_n)$: the sampled points in $\mathbb{R}^2$
- δ_j: chord lengths; λ_j: cumulative chord-length nodes
- d_j: chord vectors; t_j: unit chord directions
- H_Σ and Π_Σ: Hermite and polygonal interpolants
- $x \in [0,1]$: normalized eval. points

Motivation: Why adaptive interpolation?

Assume we know $c(t)$ we want to approximate it with a small number of interp. points. Distributing the points uniformly is not ideal.

→ So adaptive interp starts with a coarse approximation, estimates where the approximation needs improvements and inserts additional points in those regions.

We are given $c: \mathbb{R} \to \mathbb{R}^2$ and $c(t+1) = c(t)$ holds.

We begin with parameters t and evaluate $c(t)$ to obtain points $\Sigma = (p_1, \dots, p_n)$

↳ here t's are parameters for just sampling the original curve

Now we define a second parametrization for constructing the interpolating curve.

↳ $\delta_i = \begin{cases} \| p_{i+1} - p_i \|_2 & i = 1, \dots, n-1 \\ \| p_1 - p_n \|_2 & i = n \end{cases}$

chord lengths — for closing the polygon

↳ $\lambda_0 = 0$, $\lambda_k = \sum_{j=1}^{k} \delta_j$ → λ_n is total length of polygon

Unlike earlier exercise these chord-length nodes are not normalized to end at 1. The interpolants are defined on $[0, \lambda_n]$, but the function we implement receive normalized query values $x \in [0, 1)$.

⇒ For evaluation use $\lambda = x \lambda_n$

To decide where refinement is needed we construct two interpolants through Σ

A. closed polygonal interpolant Π_Σ

$$\Pi_\Sigma(\lambda) = (1 - \tau) p_j + \tau p_{j+1} \quad \text{with} \quad \tau = \frac{\lambda - \lambda_{j-1}}{\delta_j}$$

B. closed cubic Hermite interpolant H_Σ Look at what we needed for this to work from above section about Hermite.

On $[\lambda_{j-1}, \lambda_j]$ the endpoint values are p_j, p_{j+1} and endpoint derivatives are v_j, v_{j+1}

$$H_\Sigma = H_{00}(\tau) p_j + H_{01}(\tau) p_{j+1} + \delta_j H_{10}(\tau) v_j + \delta_j H_{11}(\tau) v_{j+1}$$

must find these!

How to determine which segments need refinement?

We know have two approx. through the same points Π_2, H_2.

At the interp nodes both agree, therefore comparing them at the nodes always give 0.

↳ so we compare them at the midpoint of each segment:

$$m_j = \frac{\lambda_{j-1} + \lambda_j}{2}$$

Define discrepancy $\sigma_j = \| H_2(m_j) - \Pi_2(m_j) \|_2$

Deriving useful formula for σ_j:

At the midpoint $\tau = 1/2$.

Recall $H_{00}(1/2) = H_{01}(1/2) = 1/2$

$H_{10}(1/2) = 1/8 \quad H_{11}(1/2) = -1/8$

$$\Rightarrow H_2(m_j) = \frac{P_j + P_{j+1}}{2} + \frac{\delta_j}{8}(v_j - v_{j+1})$$

$$\Rightarrow \text{And } \Pi_2(m_j) = \frac{P_j + P_{j+1}}{2}$$

$$\Rightarrow \text{So } \sigma_j = \frac{\delta_j}{8} \| v_j - v_{j+1} \|_2$$

→ discrepancies are influenced by chord-length and change in tangent direction!

Adaptive Refinement Algorithm

Initially choose parameter set $M = \left\{ 0, \frac{1}{n_{min}}, \dots \frac{n_{min}-1}{n_{min}} \right\}$

At every iteration:

1. Evaluate $c(t)$ for $t \in M$, obtaining Σ
2. Construct chord lengths, cumulative nodes and tangent estimates (v)
3. Calculate each σ_j
4. Stop if $\max \sigma_j \leq tol$, and return evaluations.
5. Compute $\alpha = \frac{1}{n} \sum_{j=1}^{n} \sigma_j$ where $n = |\Sigma|$

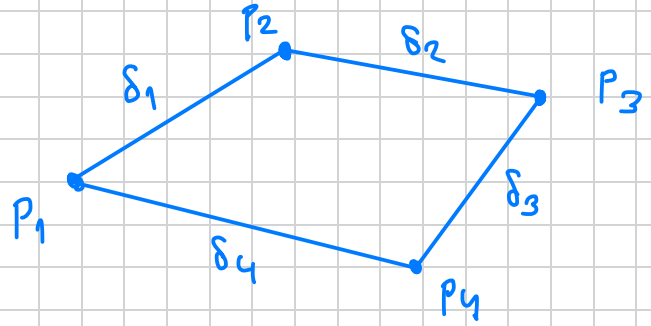
   Select segments for refinement by refining every segment with $\sigma_j > 0.9\alpha$

   ↓

   Insert new sampling points for every selected segment at the midpoint of the corr. interval in the original curve parametrization
6. Repeat until $\max \sigma_j \leq tol$ satisfied.

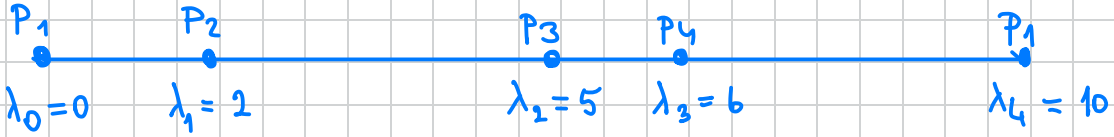
Geometric Intuition

We are given P_1, P_2, P_3, P_4 in the plane.



Suppose $\delta_1 = 2$, $\delta_2 = 3$, $\delta_3 = 1$, $\delta_4 = 4$. Total is 10.

Now imagine a distance counter that starts at zero and increases as we walk along the polygon.



In particular $\Pi_2(\lambda_0) = p_1$ and $\Pi_2(\lambda_1) = p_2$, so λ's become our interpolation nodes

On the segment connecting p_j and p_{j+1} the corr. parameter interval is $[\lambda_{j-1}, \lambda_j]$

↳ We define $\tau = \frac{\lambda - \lambda_{j-1}}{\delta_j}$ ... this is the fraction of the current polygon segment we have traversed.

What are v_j's?

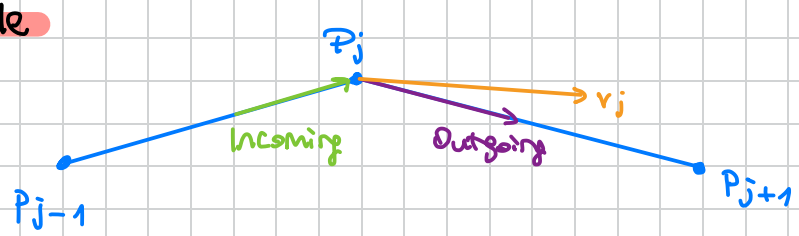
Now we want to construct a smooth curve through the points, not just a polygon → Cubic Hermite

We know we need derivatives at the interp nodes!

↳ Reminder: $s_j'(t_j) = c_j$, $s_j'(t_{j+1}) = c_{j+1}$

For our curve derivatives are 2D vectors not slopes

⇒ each v_j determines the direction in which the Hermite curve passes through a node



We estimate v_j by $t_j = \frac{p_{j+1} - p_j}{\delta_j}$ ... unit edge direction

- At a vertex we combine incoming and outgoing directions to estimate v_j